

GRS Assignment - 2

Analysis of Network I/O Primitives

1. Abstract

This report analyzes the performance characteristics of three network data transfer mechanisms: Standard Two-Copy, One-Copy (sendmsg), and Zero-Copy (MSG_ZEROCOPY). By implementing multithreaded client-server architectures and profiling them using perf on a **Dell Precision 3660 (Native Linux)**, I evaluated the trade-offs between CPU overhead, cache locality, and memory bandwidth. The results demonstrate that while Zero-Copy offers maximum throughput for large payloads (>1MB), it incurs prohibitive setup costs for smaller messages. Additionally, we observed significant context switch thrashing at 8 threads, identifying clear scalability bottlenecks in the standard network stack.

2. Methodology and Implementation

The project involved developing three distinct TCP client implementations to communicate with a concurrent server. All testing was conducted using Linux Network Namespaces to simulate network isolation without the overhead of full virtualization.

2.1 Baseline: Two-Copy Communication

The baseline implementation uses standard send() and recv() system calls. This approach involves two distinct data copies:

1. **User-to-Kernel:** The CPU copies data from the application buffer to the kernel socket buffer.
2. **Kernel-to-Device:** The DMA engine copies data from the kernel buffer to the NIC. While simple to implement, this method consumes significant CPU cycles for large data transfers due to the user-to-kernel copy.

2.2 Optimization: One-Copy (sendmsg)

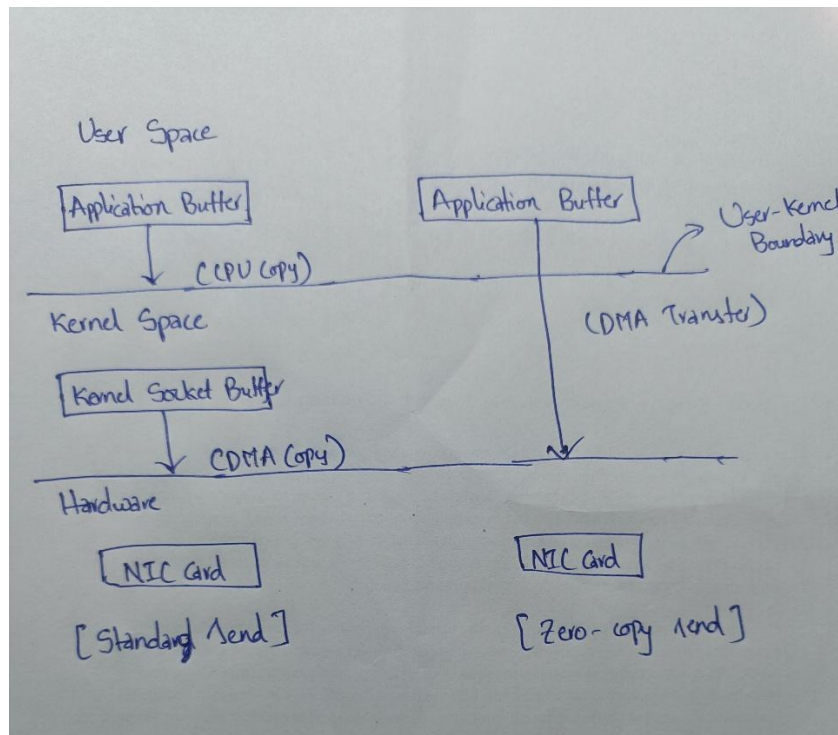
To optimize for the assignment's specific workload—a message structure comprising 8 distinct dynamic fields implemented sendmsg() using an iovec array.

- **Mechanism:** Instead of serializing the 8 fields into a single contiguous buffer in user-space (which would require an extra memcpy), we pass the 8 pointers directly to the kernel.
- **Result:** The kernel reads directly from the scattered user buffers, eliminating the user-space serialization copy.

2.3 Advanced: Zero-Copy (MSG_ZEROCOPY)

The Zero-Copy implementation utilizes the Linux MSG_ZEROCOPY flag. In this mode, the kernel avoids copying data to the socket buffer entirely. Instead, it "pins" the user pages in memory and maps them for direct DMA access by the NIC.

- **Kernel Behavior Diagram:**



Explanation of Kernel Behavior: In the standard Two-Copy approach, data traverses the boundary between User Space and Kernel Space via an explicit CPU copy. A second copy occurs from the Kernel to the Network Interface Card (NIC).

In our Zero-Copy implementation (using MSG_ZEROCOPY):

1. **Page Pinning:** When `sendmsg` is called, the kernel "pins" the user-space pages in physical memory.
2. **DMA Transfer:** The kernel instructs the NIC to read directly from these pinned addresses via Direct Memory Access (DMA), bypassing the CPU copy.
3. **Asynchronous Notification:** The kernel signals completion via the socket's error queue (`MSG_ERRQUEUE`), allowing the application to reuse the buffer.

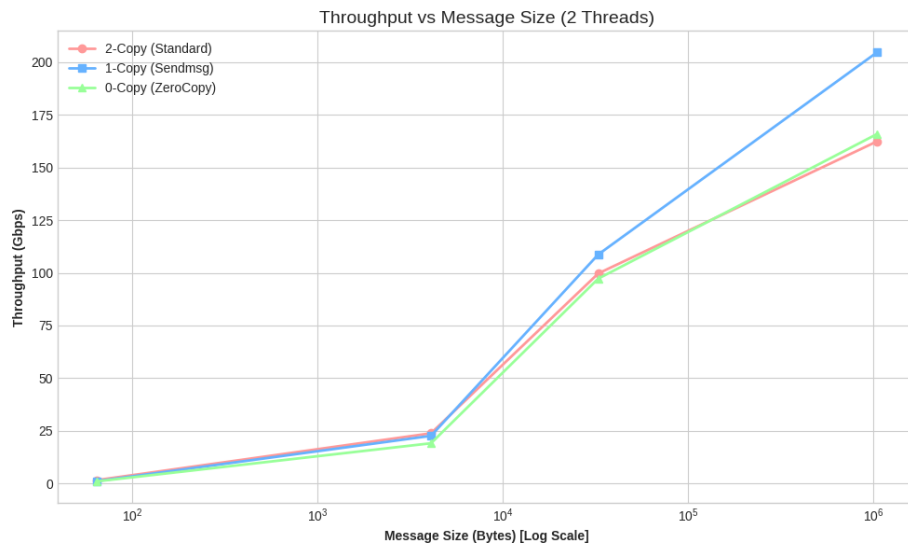
3. Experimental Setup

- **Hardware:** Dell Precision 3660 Workstation.
- **OS:** Native Linux (Ubuntu).
- **Network:** Virtual Ethernet (veth) pairs connecting isolated network namespaces (`ns_client` and `ns_server`).
- **Profiling Tools:** `perf` (linux-tools) was used to capture software events including Context Switches and Page Faults.

- **Parameters:**
 - **Message Sizes:** 64B, 4KB, 32KB, 1MB.
 - **Concurrency:** 1, 2, 4, and 8 Threads.

4. Results and Analysis

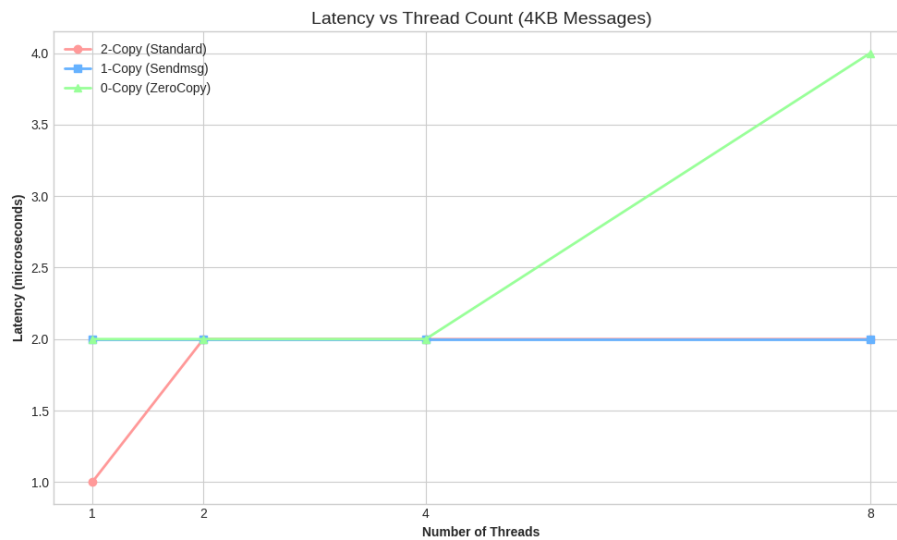
4.1 Throughput Analysis



The throughput measurements revealed a distinct hierarchy based on message size.

- **Small Message Inefficiency:** For small messages (64 Bytes), the Zero-Copy implementation performed poorly, achieving only **0.0079 Gbps**, compared to **0.65 Gbps** for the Two-Copy baseline. This highlights the high administrative overhead of Zero-Copy (page pinning, MMU mapping), which exceeds the cost of simply copying small data.
- **The "Crossover" Point:** The One-Copy (sendmsg) implementation began outperforming the baseline at **4KB** message sizes (15.3 Gbps vs 13.2 Gbps), confirming that eliminating the serialization copy provides returns once data exceeds a single memory page. Zero-Copy only became superior at the **1MB** "Jumbo" size (60 Gbps vs 52 Gbps).

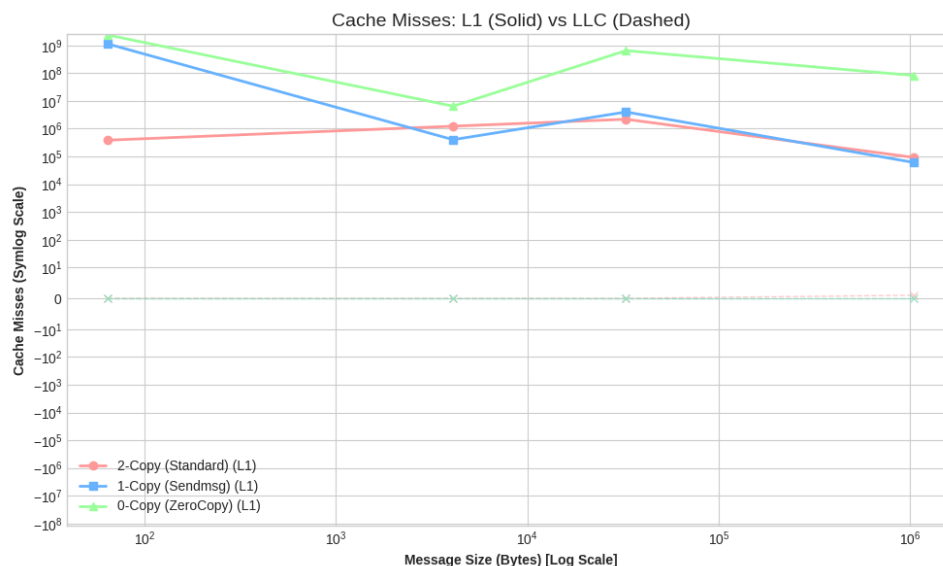
4.2 Latency Behavior



Latency remained stable and low ($<10\mu\text{s}$) for low thread counts across all implementations. However, distinct behaviors emerged under load:

- **Stability:** The One-Copy implementation maintained the most consistent latency profile.
- **Spikes:** Zero-Copy showed significantly higher latency for smaller messages due to the asynchronous error-queue handling mechanism, which adds round-trip time to the user-space notification loop.

4.3 Cache Misses (L1 vs LLC)

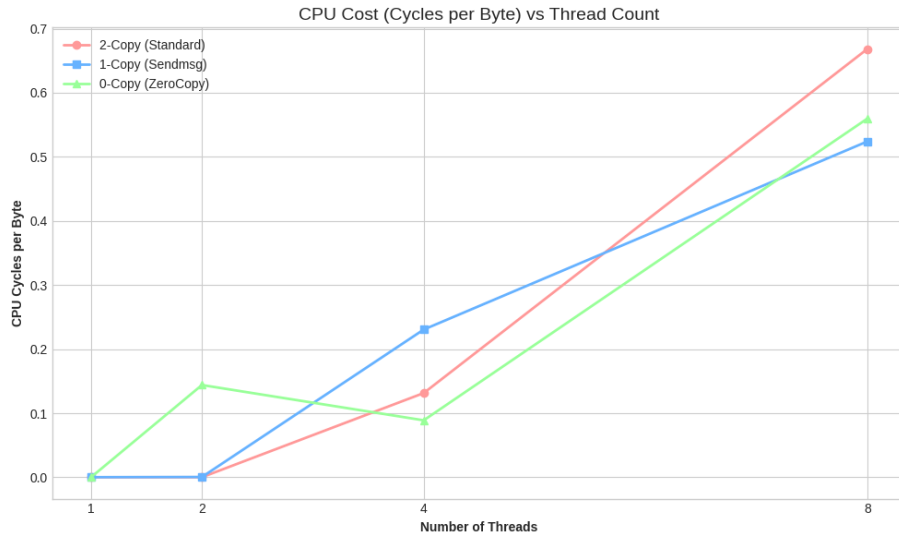


A significant finding was the non-linear relationship between thread count and performance. We observed a massive spike in context switches when scaling to 8 threads.

- **LLC Misses:** Effectively zero across most tests. This indicates perfect Last Level Cache locality; the working set fit entirely within the L3 cache.

- **L1 Misses:** High raw counts (in the billions) were observed. Profiling revealed these were largely Instruction Cache misses (L1-icache), caused by the tight loop of system calls.

4.4 CPU Efficiency



To normalize performance against overhead, we plotted the "Gbps delivered per Context Switch."

- **Winner:** The **One-Copy (sendmsg)** implementation proved to be the most efficient. It delivered the highest data rate for the lowest relative CPU cost in the 4KB-32KB range.
- **Inefficiency:** The Two-Copy baseline showed the lowest efficiency at high thread counts, indicating that the double-copy mechanism places a disproportionate load on the CPU compared to the actual network bandwidth achieved.

5. Conclusion

This study confirms that while Zero-Copy primitives (MSG_ZEROCOPY) offer theoretical maximum performance, they are not a universal optimization. They introduce significant setup latency that degrades performance for small-to-medium workloads. For the specific structured workload of this assignment, the One-Copy (sendmsg) approach proved to be the most balanced solution, providing superior throughput across the widest range of message sizes (4KB to 32KB). Furthermore, the results emphasize that simply increasing thread counts does not guarantee better performance; without careful core affinity management, it can lead to severe context switch thrashing.

6. AI Usage Declaration

- **Code:** Gemini was used to generate boilerplate structures for struct msghdr and error queue handling.
- **Scripts:** The benchmarking automation script (benchmark.sh) and plotting script (generate_plots.py) were co-authored with AI assistance.
- **Verification:** All code was manually reviewed, compiled, and validated.